РЕФЕРАТ

На программу для ЭВМ и базы данных: «Электронный образовательный ресурс по стереометрии с элементами геймификации «StereometryOnline»».

Авторы: Хамидов Абдулазиз Алишек угли

Правообладатели: Хамидов Абдулазиз Алишек угли

Дата создания: 16 май 2026 г.

Аннотация:

Назначение. Основное назначение образовательной платформы «StereometryOnline» – обеспечение эффективного процесса изучения стереометрии школьниками и студентами. Цель — сделать процесс обучения интерактивным, наглядным и мотивирующим за счет использования современных веб-технологий и игровых механик.

Область применения. Образовательные учреждения (школы, колледжи, вузы), центры дополнительного образования, платформы для дистанционного обучения.

Функциональные возможности. Программа написана на языке программирования JavaScript с использованием платформы Node.js и фреймворка Express.js. Веб-приложение предназначено для работы в браузерах под управлением ОС Windows, MacOS, Linux, Android и iOS.

Основные функции включают:

1. Система авторизации и аутентификации пользователей с использованием JWT и шифрования паролей.

2. Просмотр обучающих видеоуроков по различным разделам стереометрии (призмы, пирамиды и др.) с поддержкой потокового видео (HLS).

3. Интерактивная система заданий (теоретических и практических) с возможностью отправки решений.

4. Модуль геймификации: начисление баллов за просмотр видео и выполнение заданий, система достижений (ачивок).

5. Личный кабинет пользователя с визуализацией прогресса обучения и полученных наград.

6. Панель администратора для управления контентом, пользователями и проверки заданий.

7. Система комментариев к видеоурокам для взаимодействия учащихся и преподавателя.

Тип ЭВМ: Персональный компьютер (Windows, Mac, Linux), мобильные устройства (Android, iOS).

Язык программирования: JavaScript (ES6+), HTML5, CSS3.

Операционная система: Любая ОС с поддержкой современного веб-браузера (Chrome, Firefox, Safari, Edge).

Объем программы с базой данных: 15.2 МБ.

КОД ПРОГРАММЫ «StereometryOnline»

```
--- server.js ---

const express = require('express');

const path = require('path');

const cors = require('cors');

const { db, init } = require('./src/db');

const authRoutes = require('./src/routes/auth');

const videoRoutes = require('./src/routes/videos');

const assignmentRoutes = require('./src/routes/assignments');

const profileRoutes = require('./src/routes/profile');

const sectionRoutes = require('./src/routes/sections');

const adminRoutes = require('./src/routes/admin');


const app = express();

app.use(cors());

app.use(express.json());
```

```
app.use(express.static(__dirname));

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'index.html'));
});

app.use('/api/auth', authRoutes);
app.use('/api/videos', videoRoutes);
app.use('/api/assignments', assignmentRoutes);
app.use('/api/profile', profileRoutes);
app.use('/api/sections', sectionRoutes);
app.use('/api/admin', adminRoutes);

const port = process.env.PORT || 3000;

init().then(() => {
  if (process.env.NODE_ENV !== 'production') {
    app.listen(port, () => {
      console.log(`http://localhost:${port}/`);
    });
  }
}).catch(err => {
  console.error('Database initialization failed:', err);
});

--- src/db.js ---
const path = require('path');
const bcrypt = require('bcryptjs');
require('dotenv').config();
```

```
let db;
// ... (database connection logic and table creation)
async function init() {
  const usersTable = 'CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL, role TEXT NOT NULL DEFAULT "student", name TEXT, points INTEGER DEFAULT 0, created_at TEXT DEFAULT CURRENT_TIMESTAMP)';
  // ... (creation of other tables: sections, videos, assignments, progress, achievements, user_achievements, comments)
  await db.exec(usersTable);
  // ... (admin account seeding and initial data seeding)
}

--- src/routes/auth.js ---
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { db } = require('../db');

router.post('/register', async (req, res) => {
  const { email, password, name } = req.body;
  if (!email || !password) return res.status(400).json({ error: 'invalid' });
  try {
    const hash = await bcrypt.hash(password, 10);
    const result = await db.run('INSERT INTO users (email, password_hash, name) VALUES (?, ?, ?)', [email, hash, name]);
    const token = jwt.sign({ id: result.lastInsertRowid, email }, process.env.JWT_SECRET || 'secret');
```

```
    res.json({ token });
  } catch (e) {
    if (e.message.includes('UNIQUE')) return res.status(400).json({ error: 'exists' });
    res.status(500).json({ error: 'server_error' });
  }
});

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await db.get('SELECT * FROM users WHERE email = ?', [email]);
  if (!user) return res.status(404).json({ error: 'not_found' });
  const valid = await bcrypt.compare(password, user.password_hash);
  if (!valid) return res.status(401).json({ error: 'invalid' });
  const token = jwt.sign({ id: user.id, email: user.email, role: user.role },
process.env.JWT_SECRET || 'secret');
  res.json({ token, role: user.role });
});

--- src/utils/gamification.js ---
const { db } = require('../db');

async function awardPoints(userId, points) {
    await db.run('UPDATE users SET points = points + ? WHERE id = ?', [points,
userId]);
    await checkAchievements(userId);
}

async function checkAchievements(userId) {
    const user = await db.get('SELECT points FROM users WHERE id = ?',
[userId]);
```

```
    const watchedCountRes = await db.get('SELECT COUNT(*) as c FROM
progress WHERE user_id = ? AND status = "watched"', [userId]);
    const watchedCount = watchedCountRes.c;

    const achievements = await db.all('SELECT * FROM achievements');
    for (const ach of achievements) {
        // Logic to award achievements based on points or watched count
        // ...
    }
}

--- app.js ---
const API_BASE = '/api';

async function api(path, method = 'GET', body) {
  const headers = { 'Content-Type': 'application/json' };
  const t = localStorage.getItem('token');
  if (t) headers['Authorization'] = 'Bearer ' + t;
  const r = await fetch(API_BASE + path, { method, headers, body: body ?
JSON.stringify(body) : undefined });
  // ... (error handling and JSON parsing)
}

async function handleLogin(e) {
  e.preventDefault();
  const email = document.getElementById('loginEmail').value;
  const pass = document.getElementById('loginPass').value;
  const r = await api('/auth/login', 'POST', { email, password: pass });
  localStorage.setItem('token', r.token);
```

```
  window.location.href = '/profile.html';
}


// ... (other frontend functions: handleRegister, loadVideo, renderComments,
loadProfile)
```